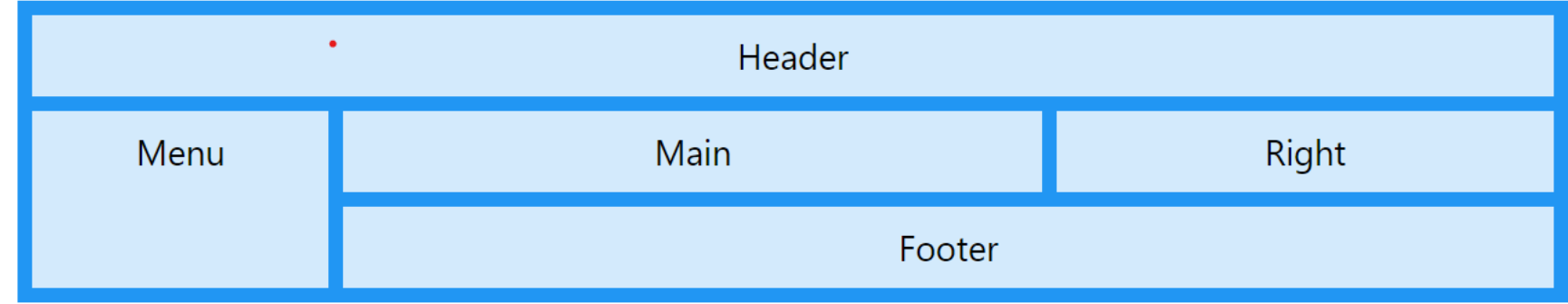

CSS Grid

Ref: <https://www.w3schools.com/> and
other internet based tutorials



Grid Layout

- The CSS Grid Layout Module offers a grid-based layout system, with rows and columns, making it easier to design web pages
 - Grid Elements
- A grid layout consists of a parent element, with one or more child elements
- Example

```
<div class="grid-container">  
  <div class="grid-item">1</div>  
  <div class="grid-item">2</div>  
  <div class="grid-item">3</div>  
</div>
```

`<style>`

`.grid-container {`

`display: grid;`

`grid-template-columns: auto auto auto;`

`background-color: #2196F3;`

`padding: 10px;`

`}`

`.grid-item {`

`background-color: rgba(255, 255, 255, 0.8);`

`border: 1px solid rgba(0, 0, 0, 0.8);`

`padding: 20px;`

`font-size: 30px;`

`text-align: center;`

`}`

`</style>`

■ Display Property

- An HTML element becomes a grid container when its display property is set to grid or inline-grid.

■ Example

```
.grid-container {  
  display: grid;  
}
```

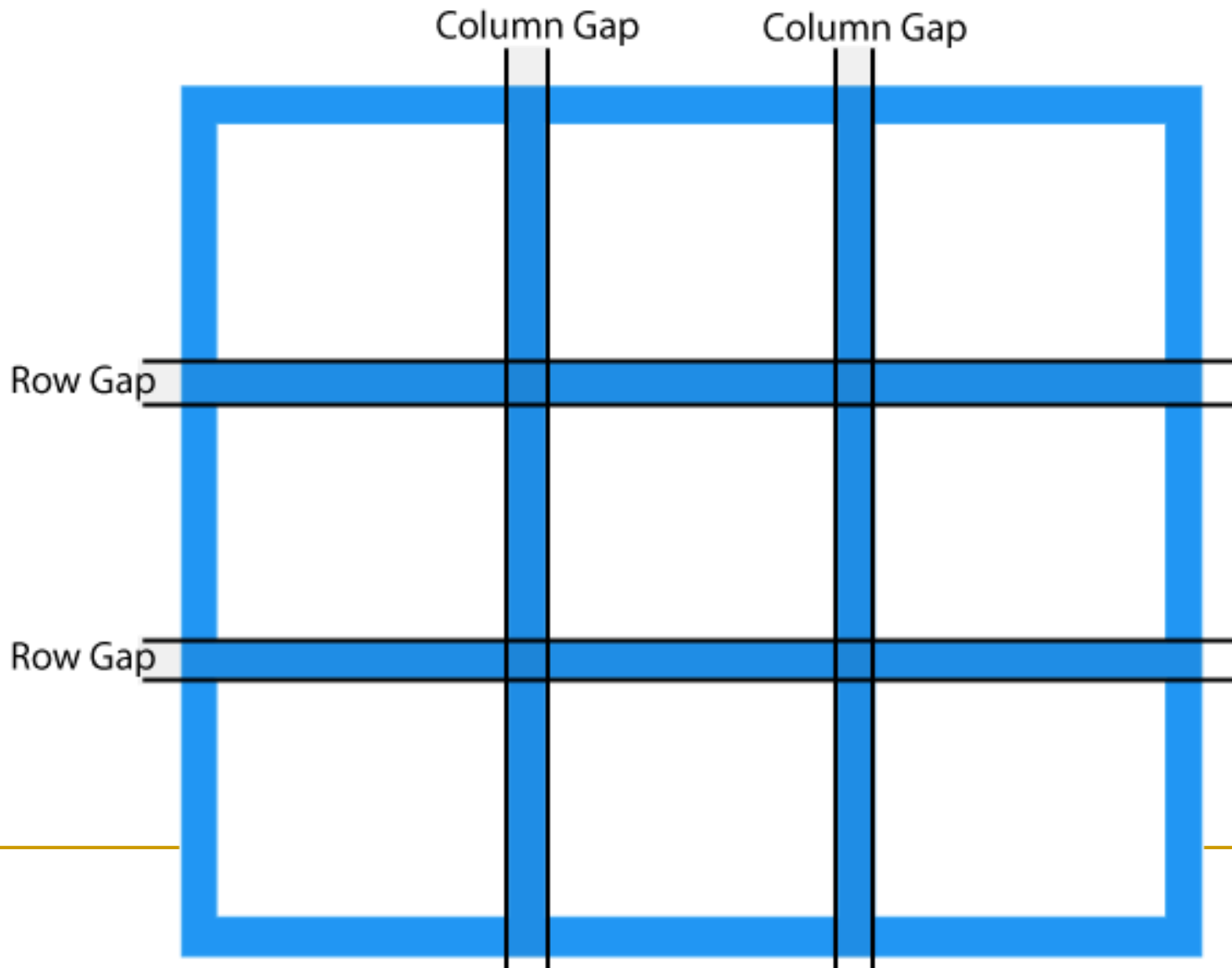
■ Example

```
.grid-container {  
  display: inline-grid;  
}
```

- All direct children of the grid container automatically become *grid items*.

Grid Gaps

The spaces between each column/row are called *gaps*.



Grid Gaps

- You can adjust the gap size by using one of the following properties:
 - ❑ `grid-column-gap`
 - ❑ `grid-row-gap`
 - ❑ `grid-gap`

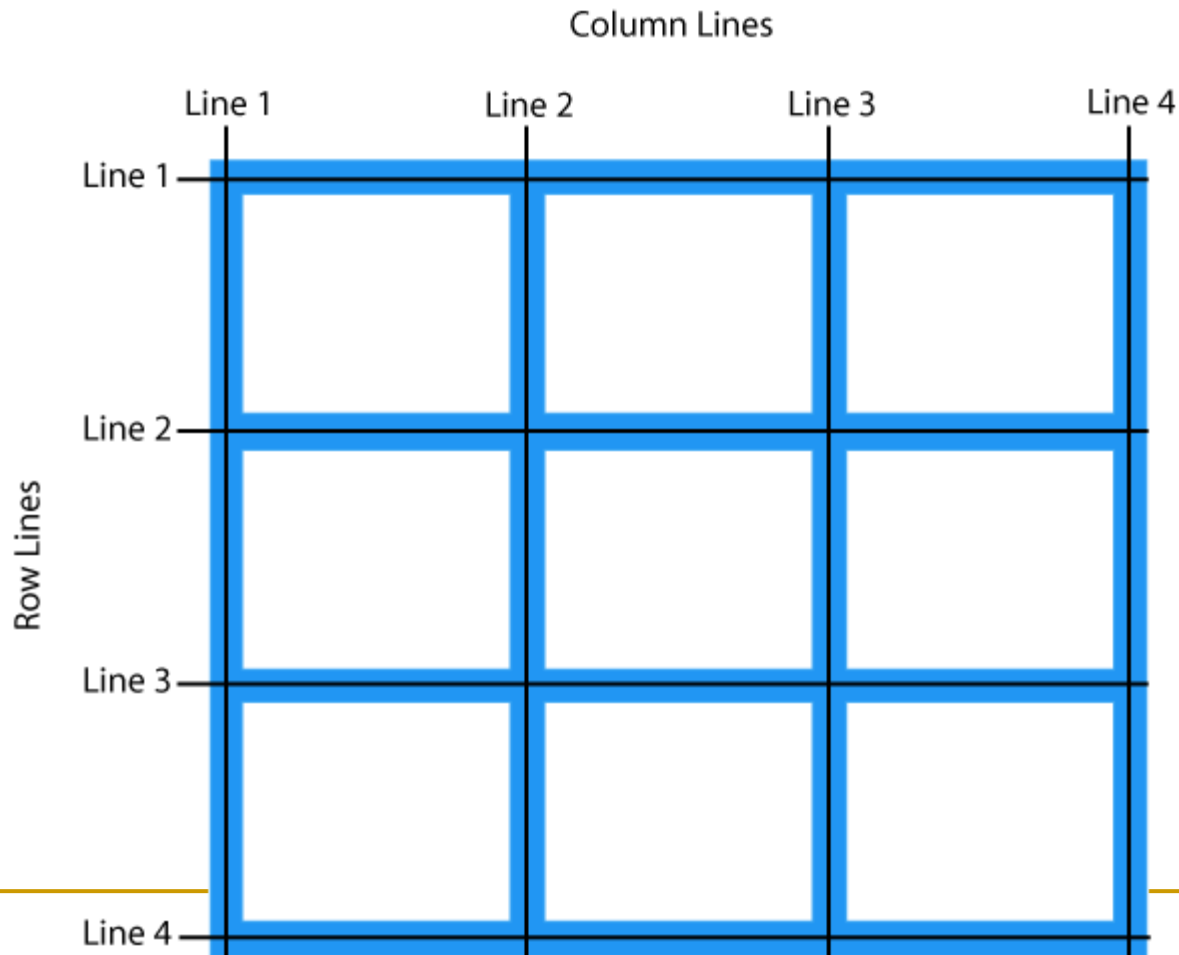
```
.grid-container {  
  display: grid;  
  grid-column-gap: 50px;  
}
```

```
.grid-container {  
  display: grid;  
  grid-row-gap: 50px;  
}
```

```
.grid-container {  
  display: grid;  
  grid-gap: 50px 100px;  
}
```

Grid Lines

- The lines between columns are called column lines.
- The lines between rows are called row lines.



Grid Lines

- Example

- Place a grid item at column line 1, and let it end on column line 3:

```
.item1 {  
  grid-column-start: 1;  
  grid-column-end: 3;  
}
```

- Example

- Place a grid item at row line 1, and let it end on row line 3:

```
.item1 {  
  grid-row-start: 1;  
  grid-row-end: 3;
```

```
}
```

grid-template-columns

- The grid-template-columns property defines the number of columns in your grid layout, and it can define the width of each column.
- If you want your grid layout to contain 4 columns, specify the width of the 4 columns, or "auto" if all columns should have the same width.
- Example
- Make a grid with 4 columns:

```
.grid-container {  
  display: grid;  
  grid-template-columns: auto auto auto auto;  
}
```

Note: *If you have more than 4 items in a 4 columns grid, the grid will automatically add a new row to put the items in.*

grid-template-columns

- The `grid-template-columns` property can also be used to specify the size (width) of the columns.
- Example
- Set a size for the 4 columns:

```
.grid-container {  
  display: grid;  
  grid-template-columns: 80px 200px auto 40px;  
}
```

grid-template-rows Property

- The grid-template-rows property defines the height of each row.
- The value is a space-separated-list, where each value defines the height of the respective row:
- Example

```
.grid-container {  
  display: grid;  
  grid-template-rows: 80px 200px;  
}
```

CSS Grid Item

■ Child Elements (Items)

- A grid container contains grid items.
- By default, a container has one grid item for each column, in each row, but you can style the grid items so that they will span multiple columns and/or rows.
- The grid-column Property:
 - The grid-column property defines on which column(s) to place an item.
 - You define where the item will start, and where the item will end.
 - To place an item, **you can refer to line numbers, or use the keyword "span"** to define how many columns the item will span.
 - Example: make "item1" start on column 1 and end on column 5:

```
.item1 {  
  grid-column: 1 / 5;}  
}
```

Note: The *grid-column* property is a shorthand property for the *grid-column-start* and the *grid-column-end* properties.

CSS Grid Item

- Example
- Make "item1" start on column 1 and span 3 columns:
*.item1 {
 grid-column: 1 / span 3;
}*
- Example
- Make "item2" start on column 2 and span 3 columns:
*.item2 {
 grid-column: 2 / span 3;
}*

■ The grid-row Property:

- The grid-row property defines on which row to place an item.
- You define where the item will start, and where the item will end.
- To place an item, you can refer to line numbers, or use the keyword "span" to define how many rows the item will span:
- Example: make "item1" start on row-line 1 and end on row-line 4:

```
.item1 {  
  grid-row: 1 / 4;  
}
```

```
.item1 {  
  grid-row: 1 / span 3;  
}
```

■ The grid-area Property

- The grid-area property can be used as a shorthand property for the grid-row-start, grid-column-start, grid-row-end and the grid-column-end properties.
- Example: make "item8" start on row-line 1 and column-line 2, and end on row-line 5 and column line 6:

```
.item8 {  
  grid-area: 1 / 2 / 5 / 6;  
}
```

- Example: make "item8" start on row-line 2 and column-line 1, and span 2 rows and 3 columns:

```
.item8 {  
  grid-area: 2 / 1 / span 2 / span 3;  
}
```


Naming Grid Items

- The grid-area property can also be used to assign names to grid items.
- Named grid items can be referred to by the grid-template-areas property of the grid container.
- Example :Item1 gets the name "myArea" and spans all five columns in a five columns grid layout:

```
.item1 {
```

```
  grid-area: myArea;
```

```
}
```

```
.grid-container {
```

```
  grid-template-areas: 'myArea myArea myArea myArea myArea';
```

```
}
```

-
- Each row is defined by apostrophes (' ')
 - The columns in each row is defined inside the apostrophes, separated by a space.
 - *Note: A period sign represents a grid item with no name.*
 - **To define two rows**, define the column of the second row inside another set of apostrophes:
 - Example: make "item1" span two columns and two rows:

```
.grid-container {  
  grid-template-areas: 'myArea myArea . . .' 'myArea myArea . . .';  
}
```
-